

TESTE DE PERFORMANCE 4



Engenharia de Software

Projeto de Bloco: Engenharia de Softwares Escaláveis

Aluna: Rafaela Oliveira Ribeiro

11/09/2026

Refatoração para arquitetura orientada a eventos

Até a etapa anterior o sistema tinha dois pontos de acoplamento síncrono. O monólito chamava o categoria-service por um Feign Client toda vez que precisava exibir o nome da categoria de um flashcard, e o histórico de alterações era gravado dentro da mesma transação do CRUD. Nesta etapa esses dois pontos foram substituídos por comunicação assíncrona, com o RabbitMQ atuando como message broker e o Spring AMQP cuidando da integração do lado do Spring Boot.

O broker sobe por docker compose na porta 5672, com painel de administração na porta 15672. O monólito continua na porta 8080 e o categoria-service na porta 8081, cada um com o seu banco H2 em arquivo e sem nenhum acesso direto ao banco do outro, como já acontecia antes.

Prós e contras da arquitetura orientada a eventos

O ganho mais visível no projeto foi o desacoplamento temporal. O monólito não depende mais de o categoria-service estar no ar no momento exato da leitura, porque responde usando o último dado conhecido guardado no cache local. Também ficou mais fácil escalar e evoluir cada serviço no seu próprio ritmo, e qualquer serviço novo que precise saber quando uma categoria muda basta declarar a sua fila ligada ao exchange categoria.events, sem que o produtor seja alterado.

Em troca, o sistema passou a conviver com consistência eventual no lugar de consistência forte, ou seja, existe uma janela de tempo em que o cache de categorias ou o histórico podem estar atrasados em relação à origem. Isso é aceitável aqui porque não é dado crítico, mas precisa ser avaliado caso a caso. A ordem de inicialização também passou a importar, porque uma mensagem publicada antes de a fila do monólito existir acaba se perdendo, e isso apareceu durante os testes com as categorias de exemplo criadas pelo DataInitializer. A mitigação adotada foi subir primeiro o broker, depois o categoria-service e por último o monólito, já que a fila é durável e sobrevive a reinícios do monólito.

Também não existe garantia transacional entre salvar no banco e publicar o evento, então uma falha bem no meio dos dois passos faz o evento nunca sair. Isso foi assumido por causa do escopo da disciplina, e num sistema em produção o tratamento seria um

Transactional Outbox no produtor ou Change Data Capture, junto com um endpoint de reconciliação para ressincronizar o cache.

O quadro abaixo resume em que situações a troca por eventos compensa e em que situações a chamada síncrona ainda é a melhor escolha, mesmo num sistema com vários serviços.

Situação	Abordagem mais adequada
O consumidor não precisa de resposta imediata, como na gravação do histórico	Eventos, com fila de trabalho
O dado de outro serviço é lido muito mais do que escrito, como o nome da categoria	Eventos, com réplica local de leitura
O serviço precisa continuar operando com dependências fora do ar	Eventos, aceitando dado levemente atrasado
A resposta do outro serviço decide o que fazer agora, como autorizar um pagamento	Chamada síncrona
Poucos serviços e prioridade para a simplicidade de depuração	Chamada síncrona

Padrões de mensagem implementados

O primeiro padrão é publish/subscribe com topic exchange, no formato conhecido como event-carried state transfer. O categoria-service publica um evento a cada escrita no exchange categoria.events, usando as routing keys categoria.criada, categoria.atualizada e categoria.deletada. O monólito assina esses eventos pela fila monolito.categoria.sync.queue, com binding categoria.*, e mantém a tabela categoria_cache como réplica de leitura. Assim o campo categoriaNome é preenchido sem nenhuma chamada de rede no momento da leitura. A escolha do topic exchange no lugar de um fanout foi pensando nos consumidores futuros, que poderão assinar apenas parte dos eventos, como um serviço de limpeza interessado somente em categoria.deletada.

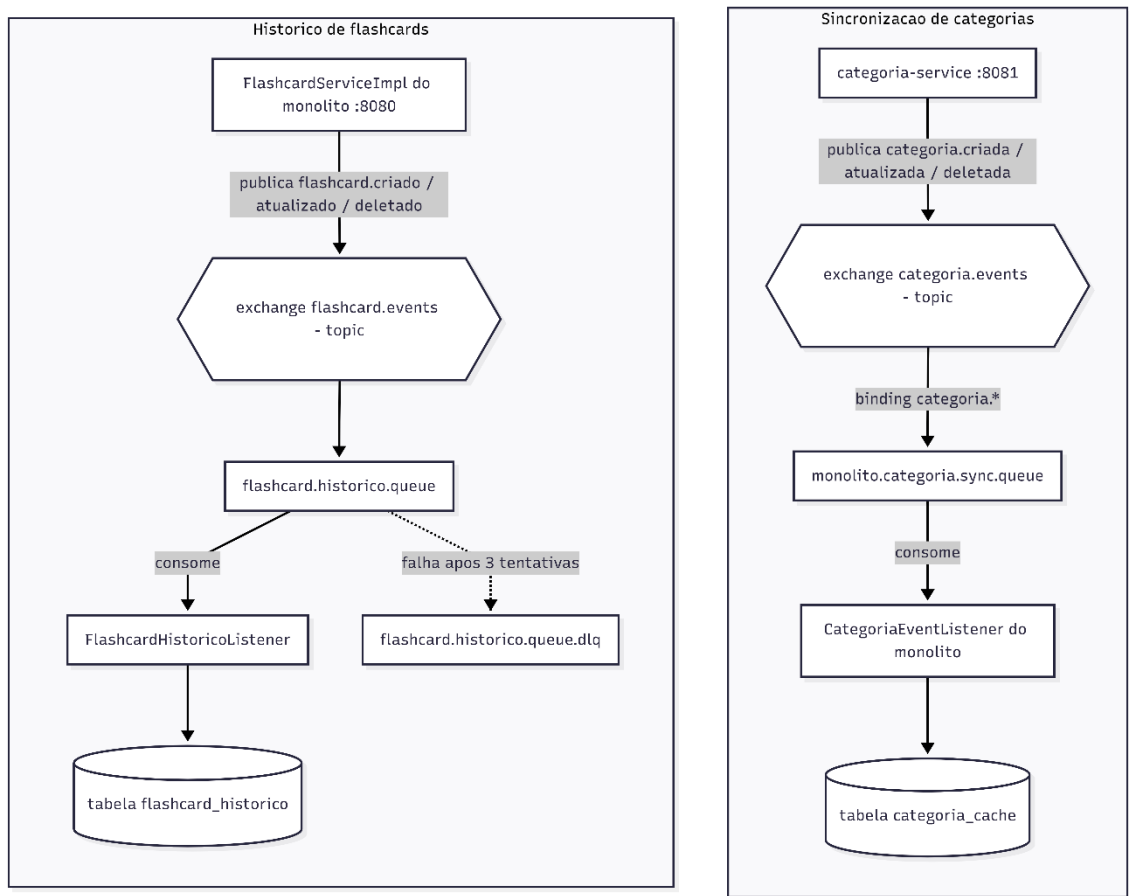
O segundo padrão é uma fila de trabalho assíncrona com retry e dead letter queue, usada para tirar a gravação do histórico de dentro da transação do CRUD. O próprio monólito publica no exchange flashcard.events com as routing keys flashcard.criado, flashcard.atualizado e flashcard.deletado, e consome a fila flashcard.historico.queue,

que tem os argumentos x-dead-letter-exchange e x-dead-letter-routing-key apontando para a fila flashcard.historico.queue.dlq. O listener tenta gravar até três vezes antes de a mensagem ser rejeitada em definitivo, o que evita perder o evento por uma falha passageira do banco sem ficar reprocessando para sempre uma mensagem com defeito.

Padrão	Exchange e routing keys	Produtor e consumidor
Pub/Sub com topic exchange, sincronizando o nome das categorias	categoria.events, com categoria.criada, categoria.atualizada e categoria.deletada	Produzido pelo categoria-service e consumido pelo monólito na fila monolito.categoria.sync.queue
Fila de trabalho com retry e DLQ, gravando o histórico fora da transação	flashcard.events, com flashcard.criado, flashcard.atualizado e flashcard.deletado	Produzido e consumido pelo próprio monólito na fila flashcard.historico.queue

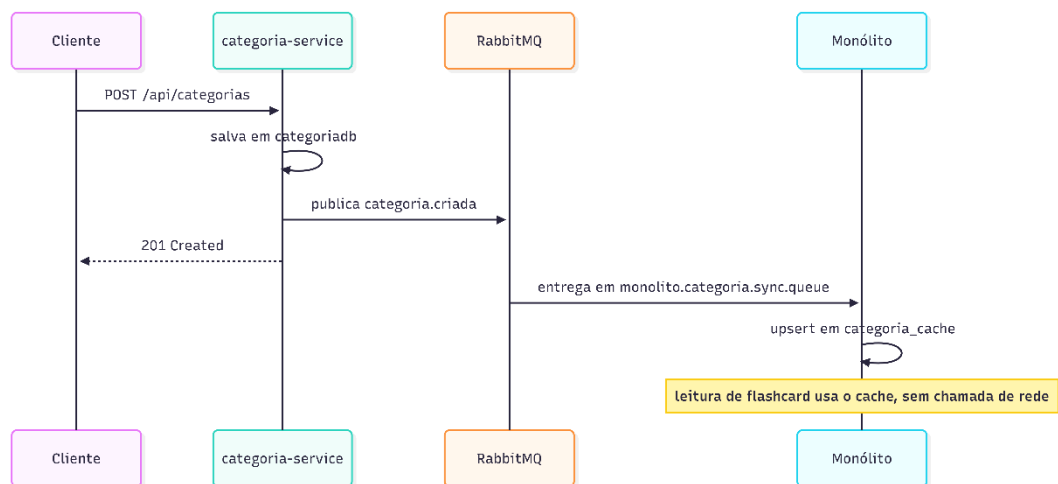
Diagrama da arquitetura

O desenho abaixo mostra como os serviços passaram a conversar depois da refatoração.

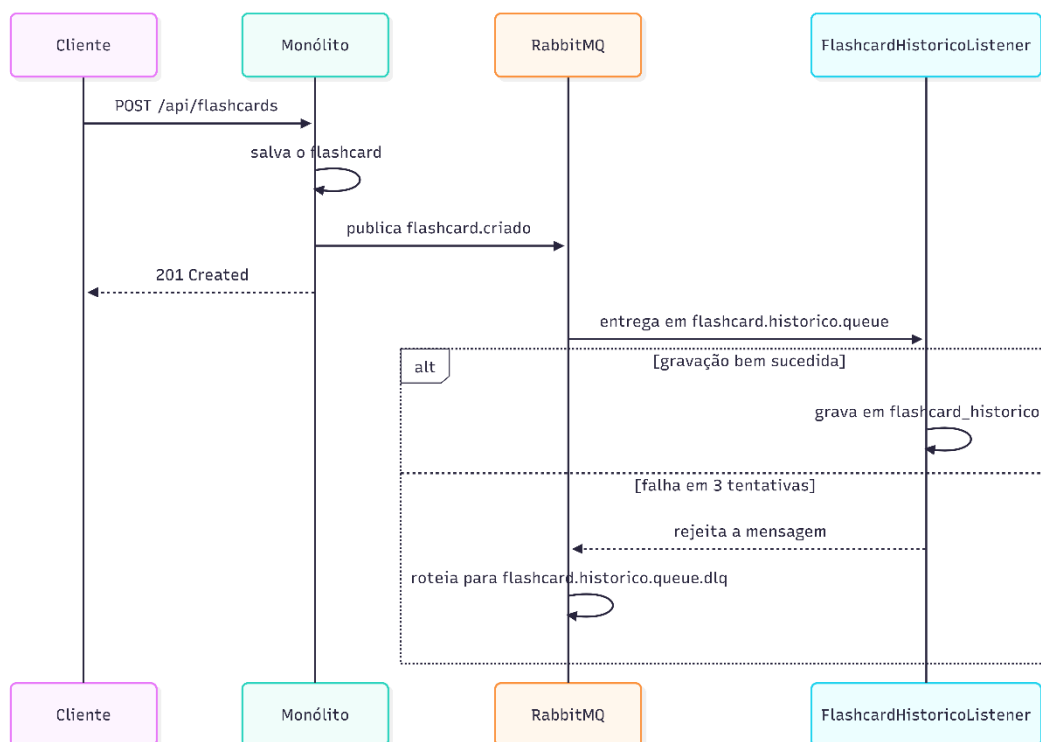


Fluxos de evento

Sincronização de uma categoria:



Gravação do histórico de um flashcard:



Implementação com Spring Boot

Os dois serviços receberam a dependência `spring-boot-starter-amqp` e a configuração de conexão com o broker no `application.properties`. Em cada projeto uma classe de configuração declara os exchanges, as filas e os bindings como beans, então a topologia

é criada sozinha quando a aplicação sobe e não é preciso montar nada à mão no painel do RabbitMQ. As mensagens trafegam em JSON, com um conversor registrado no RabbitTemplate e no container dos listeners, de modo que produtor e consumidor trocam objetos de evento e não texto solto.

Do lado de quem publica, o CategoriaEventPublisher e o FlashcardServiceImpl usam o RabbitTemplate logo depois do save. Do lado de quem consome, o CategoriaEventListener e o FlashcardHistoricoListener são métodos anotados com @RabbitListener, e o Spring entrega a mensagem já desserializada. O retry do histórico é ligado por propriedade, em spring.rabbitmq.listener.simple.retry, sem nenhum código extra.

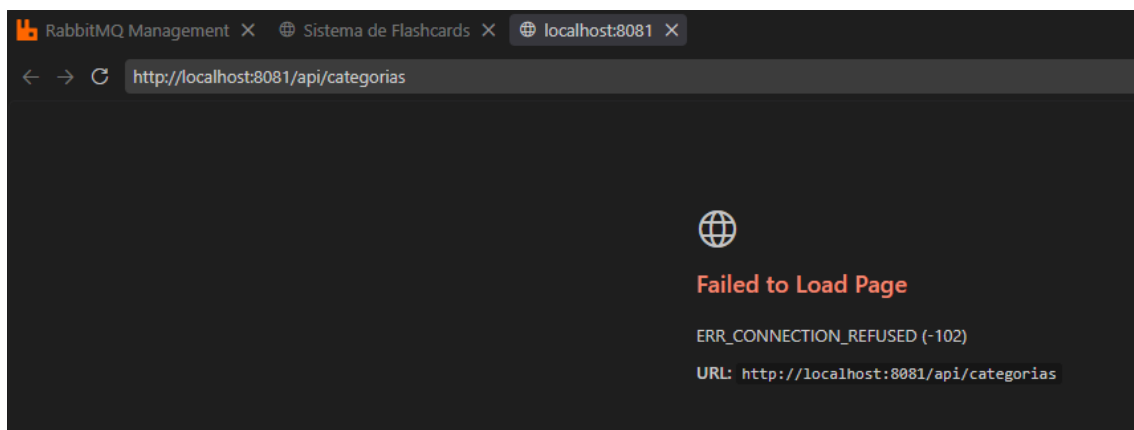
Os DTOs de evento ficaram duplicados nos dois projetos, sem módulo Java compartilhado, para manter os dois builds Maven independentes, como já era o caso do DTO usado pelo Feign na etapa anterior.

Evidências

Overview					Messages			Message rates				+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver	/ get	ack	
/	flashcard.historico.queue	classic	D DLX DLK	running	0	0	0	0.20/s		0.20/s	0.20/s	
/	flashcard.historico.queue.dlq	classic	D	running	0	0	0					
/	monolito.categoria.sync.queue	classic	D	running	0	0	0	0.00/s		0.00/s	0.00/s	

► Add a new queue

1: o categoria-service (porta 8081) está fora do ar.



2: mesmo assim, o flashcard continua exibindo o nome da categoria normalmente.

PERGUNTA

Alinhamento estratégico

Editar

RESPOSTA

garante que os planos de tecnologia caminhem junto com as metas do negócio

Excluir

→ Governança de TI

Isso mostra que o monólito não depende mais de uma chamada em tempo real ao categoria-service, porque o nome da categoria já estava salvo localmente, pelo evento do RabbitMQ, antes de o serviço cair.

Link do Repositório Github

<https://github.com/RafaelaRibe1ro/flashcards-sistema>